

# Anime Finder App Summary (One Page)

Repo analyzed: Flutter client + FastAPI backend + Docker services

## What It Is

Anime Finder (app title shown as **Anime Seek**) is a cross-platform Flutter app with a FastAPI backend for anime scene and audio recognition, plus account and community features. The repo includes client code, backend API code, and containerized infrastructure for Solr and MariaDB-backed search.

## Who It's For

Primary persona: anime fans who want to identify scenes/songs and track/share discoveries in-app. **Explicit product persona documentation: Not found in repo.**

## What It Does

- Scene search from gallery image or URL via `/search`, with top matches, similarity scores, and preview clips/images.
- Audio fingerprint recognition flow (`/recognize` and `/fingerprint`) with matched anime/song metadata and match history.
- User authentication and account flows: register, token login, refresh token handling, verify/reset password, and profile updates.
- Social/feed features: global feed, likes, reshares, comments/replies, and profile follow actions.
- Favorites and personal history models for anime results and scene logs, persisted locally in app preferences.
- Subscription/payment integration signals in code (Stripe key in app, subscription endpoints, webhook module in backend).
- Optional market/wallet screens and services (`/markets`, `/me/wallet`) present in codebase.

## How It Works (Repo-Evidenced Architecture)

**Client:** Flutter app (`lib/`) using Provider state management and Dio/http clients; routes include recognition, feed, profile, notifications, and auth screens.

**APIs:** App calls `https://anime-seek.com/fastapi` and `https://api.anime-seek.com` endpoints; also queries AniList GraphQL for metadata enrichment.

**Backend:** FastAPI app (`anime\_audio\_backend/main.py`) exposes recognition/search/playlist endpoints; uses ffmpeg + Dejavu/PyDejavu for audio processing/fingerprints.

**Data:** SQLAlchemy + PyMySQL models/sessions (`database.py`, `models.py`) with MariaDB/MySQL tables for users, playlists, songs, and anime metadata.

**Infra:** `Docker-compose.yml` defines services for web/api, FastAPI, MariaDB, LIRE Solr, and static UI on a shared bridge network.

**Flow:** User submits media in app -> API search/recognize -> metadata enrichment/cache (AniList + local DB) -> results returned to client and optionally persisted.

## How To Run (Minimal Getting Started)

- 1 Create and populate `.env` for compose variables (template file for required keys: **Not found in repo**).
- 2 Start backend stack from repo root: **`docker compose -f Docker-compose.yml up --build`**.
- 3 Install Flutter deps: **`flutter pub get`**.
- 4 Run client: **`flutter run`** (or platform-specific target).
- 5 If running backend without Docker, use `anime\_audio\_backend/requirements.txt`; exact non-Docker startup procedure: **Not found in repo**.